BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

KHOA ĐIỆN – ĐIỆN TỬ

---------- ✦ ----------

BÁO CÁO

Môn: Hệ thống nhúng

ĐỀ SỐ: 4

ĐỀ BÀI: THIẾT KẾ MẠCH TỰ ĐỘNG GIÁM SÁT THÔNG SỐ MÔI TRƯỜNG CHO BỂ NUÔI THỦY, HẢI SẢN SỬ DỤNG ESP32, HC-SR04, DS18B20, FREERTOS VÀ MQTT

Giảng viên hướng dẫn: Ths. Võ Quang Sơn

Sinh viên thực hiện: Nguyễn Thế Dân

Mã sinh viên: 231432188

Lớp: Điện Tử Và Tin Học Công Nghiệp K64

Hà Nội 2026

MỤC LỤC

LỜI MỞ ĐẦU **3**
I. PHẦN MỞ ĐẦU **4**
1.1. Đặt vấn đề 4
1.2. Yêu cầu của đề bài và hướng giải quyết 4
1.3. Mục tiêu và phạm vi 5
II. CẢM BIẾN VÀ THIẾT BỊ CHẤP HÀNH **5**
2.1. Vi điều khiển ESP32 5
2.2. Cảm biến siêu âm HC-SR04 6
2.3. Cảm biến nhiệt độ DS18B20 7
2.4. Cơ cấu chấp hành: máy bơm và van thoát 7
III. HỆ ĐIỀU HÀNH THỜI GIAN THỰC RTOS VÀ FREERTOS **8**
3.1. Khái niệm RTOS 8
3.2. Giới thiệu FreeRTOS 8
3.3 FreeRTOS trên ESP32 9
IV. CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK **9**
4.1. Nhu cầu giao tiếp giữa các Task 9
4.2. Hàng đợi (Queue) 10
4.2.1. Khái niệm và nguyên lý hoạt động 10
4.2.2. Ưu điểm 11
4.2.3. Ứng dụng 11
4.3. Cờ hiệu (Semaphore) và Mutex 11
4.3.1. Khái niệm và các loại 11
4.3.2. Ưu điểm 12
4.3.3. Ứng dụng 13
4.4. Lựa chọn cơ chế giao tiếp trong bài Lab 13
V. GIAO THỨC MQTT VÀ MQTT BROKER **14**
5.1. Giới thiệu MQTT 14
Ưu điểm và ứng dụng của MQTT 15
5.2 Các MQTT Broker thông dụng 15
5.3 Lựa chọn EMQX Public Broker 16
VI.CÁC BƯỚC THỰC HIỆN BÀI LAB **16**
6.1 Sơ đồ mạch trong Wokwi 16
6.2 Triển khai FreeRTOS trên ESP32 18
6.2.1. Phân chia Task và các hàng đợi 18

6.2.2. Bảng dữ liệu của các Task và hàng đợi 19
6.3Bảng topic và cổng kết nối 20
6.4. Thuật toán điều khiển mực nước 20
6.5. Giao diện Web 21
6.6 Mã nguồn và giải thích 23
6.6.1. Chương trình ESP32 23
6.6.2. Chương trình Web 44
KẾT LUẬN 53
TÀI LIỆU THAM KHẢO 53

LỜI MỞ ĐẦU

Trong lĩnh vực nuôi trồng thủy, hải sản, việc giám sát và kiểm soát các thông số môi trường như mực nước và nhiệt độ có vai trò quan trọng trong việc duy trì điều kiện sống ổn định cho các sinh vật thủy sinh. Tuy nhiên, các phương pháp giám sát thủ công còn tồn tại nhiều hạn chế như tốn thời gian, khó theo dõi liên tục và chưa đáp ứng tốt yêu cầu tự động hóa. Vì vậy, việc ứng dụng công nghệ vi điều khiển và Internet vạn vật (IoT) vào xây dựng hệ thống giám sát, điều khiển tự động là một giải pháp có tính ứng dụng thực tiễn.

Xuất phát từ yêu cầu đó, nhóm thực hiện đề tài **"Thiết kế mạch tự động giám sát thông số môi trường cho bể nuôi thủy, hải sản"** với bộ xử lý trung tâm là ESP32. Hệ thống sử dụng cảm biến siêu âm HC-SR04 để đo khoảng cách từ cảm biến đến mặt nước, qua đó xác định mực nước trong bể, kết hợp với cảm biến nhiệt độ để thu thập dữ liệu môi trường. Các thông số sau khi xử lý được hiển thị qua giao tiếp Serial và truyền đến MQTT Broker thông qua kết nối Wi-Fi, cho phép người dùng theo dõi trực tiếp trên giao diện Web/App.

Về mặt kỹ thuật, đề tài tập trung triển khai cơ chế đa nhiệm sử dụng FreeRTOS tích hợp trên ESP32. Các chức năng được phân chia thành những Task riêng biệt, bao gồm đọc dữ liệu cảm biến, truyền dữ liệu lên MQTT Broker và xử lý cảnh báo, điều khiển cơ cấu chấp hành. Việc trao đổi dữ liệu giữa các Task được thực hiện thông qua Queue, giúp tổ chức quá trình truyền nhận dữ liệu và phối hợp hoạt động giữa các nhiệm vụ. Bên cạnh đó, hệ thống xây dựng thuật toán điều khiển tự động máy bơm nước vào và van thoát nước dựa trên các ngưỡng mực nước được thiết lập. Người dùng cũng có thể điều khiển trực tiếp các cơ cấu chấp hành thông qua các nút nhấn trên giao diện Web/App ở chế độ Manual.

Thông qua đề tài, nhóm hướng đến việc vận dụng các kiến thức về vi điều khiển ESP32, cảm biến, lập trình nhúng, hệ điều hành thời gian thực FreeRTOS và giao thức truyền thông MQTT để xây dựng một hệ thống giám sát, điều khiển có khả năng hoạt động tự động và giám sát từ xa. Qua đó, nhóm có cơ hội củng cố kiến thức chuyên môn, rèn luyện kỹ năng lập trình và tích hợp giữa phần cứng với phần mềm trong các ứng dụng IoT thực tế.

Do thời gian và kinh nghiệm thực hiện còn hạn chế, báo cáo khó tránh khỏi những thiếu sót.
Em rất mong nhận được sự góp ý và hướng dẫn của giảng viên để hoàn thiện đề tài tốt hơn.

I. PHẦN MỞ ĐẦU

1.1. Đặt vấn đề

Trong nuôi trồng thủy, hải sản theo phương pháp truyền thống, người nuôi thường kiểm tra mực nước và nhiệt độ bằng mắt thường hoặc nhiệt kế cầm tay vào những thời điểm cố định trong ngày. Cách làm này tốn nhân công, thiếu dữ liệu liên tục và không kịp thời phát hiện các biến động bất thường, nhất là vào ban đêm hoặc khi người nuôi vắng mặt.

Mực nước quá thấp làm giảm thể tích sống, nhiệt độ nước dễ dao động mạnh và nồng độ chất thải tăng nhanh; ngược lại, mực nước quá cao có nguy cơ tràn bể, thất thoát vật nuôi. Với nhiều loài tôm, cá nước lợ, khoảng nhiệt độ thích hợp thường nằm trong dải khoảng 26 – 32 °C (tùy từng loài và giai đoạn phát triển), nên việc theo dõi nhiệt độ liên tục là cần thiết.

Vì vậy cần một hệ thống tự động có khả năng: (1) đo mực nước và nhiệt độ liên tục; (2) truyền dữ liệu về trung tâm và hiển thị trực quan cho người vận hành từ xa; (3) tự động điều khiển máy bơm cấp nước và van xả nước theo thuật toán phù hợp; (4) cho phép người vận hành can thiệp thủ công khi cần. Bài Lab này xây dựng một mô hình thu nhỏ của hệ thống như vậy trên nền ESP32 và FreeRTOS.

1.2. Yêu cầu của đề bài và hướng giải quyết

Đề số 4 yêu cầu thiết kế mạch tự động giám sát thông số môi trường cho bể nuôi thủy, hải sản. Bảng dưới đây đối chiếu từng yêu cầu của đề bài với giải pháp được thực hiện trong báo cáo.

Bảng 1.1. Đối chiếu yêu cầu đề bài và giải pháp thực hiện

STT	Yêu cầu của đề bài	Giải pháp trong bài Lab
1	Vẽ mạch kết nối trong Wokwi	Mạch gồm ESP32 DevKit C v4, HC-SR04, DS18B20, 2 LED (mô phỏng bơm, van) và các điện trở
2	Đọc mực nước (HC-SR04) và nhiệt độ bể, hiển thị qua Serial	TaskReadData đọc cảm biến mỗi 1 s và in ra Serial Monitor
3	Gửi dữ liệu lên MQTT Broker qua Wi-Fi	TaskTransmitData (Core 0) kết nối Wi-Fi, publish JSON lên topic aquarium/data
4	Web/App: 2 thẻ hiển thị Mực nước, Nhiệt độ dạng số	Hai thẻ HTML class "card" với id water và temp

STT	Yêu cầu của đề bài	Giải pháp trong bài Lab
5	2 biểu đồ cập nhật thời gian thực (tùy chọn)	Hai biểu đồ đường Chart.js, tối đa 30 điểm
6	2 nút điều khiển Máy bơm vào, Van thoát ra (Manual)	Hai nút button bật/tắt, chỉ hoạt động ở chế độ MANUAL
7	Task truyền data – Core 0, đọc từ data Queue	TaskTransmitData ghim Core 0, nhận từ dataQueue
8	Task đọc data – Core 1, gửi data Queue và data Alert	TaskReadData ghim Core 1, gửi dataQueue và dataAlert
9	Task cảnh báo – Core 1, điều khiển bơm và van theo thuật toán	TaskAlert ghim Core 1 đọc dataAlert và điều khiển theo 3 vùng mực nước
10	Trình bày RTOS, FreeRTOS, semaphore, queue	Chương III và Chương IV của báo cáo

1.3. Mục tiêu và phạm vi

Bài Lab thực hiện các nội dung sau:
Thiết kế và mô phỏng mạch ESP32, HC-SR04, DS18B20 và thiết bị chấp hành trên Wokwi
-Đọc dữ liệu mực nước, nhiệt độ và hiển thị qua Serial.
-Triển khai FreeRTOS với 3 Task, các Queue trao đổi dữ liệu và phân bổ tác vụ trên 2 lõi ESP32.
-Kết nối Wi-Fi, sử dụng MQTT để truyền dữ liệu và nhận lệnh điều khiển.
-Thiết kế giao diện Web gồm 2 thẻ hiển thị thông số, 2 biểu đồ thời gian thực và các nút điều khiển máy bơm, van xả.
-Xây dựng chế độ Auto điều khiển theo ngưỡng mực nước và chế độ Manual điều khiển trực tiếp từ Web.
Trong mô phỏng, relay được sử dụng để mô phỏng hoạt động của máy bơm và van xả. Hệ thống sử dụng ESP32 để giám sát, xử lý dữ liệu và điều khiển thiết bị thông qua MQTT.

II. CẢM BIẾN VÀ THIẾT BỊ CHẤP HÀNH

2.1. Vi điều khiển ESP32

ESP32 là dòng vi điều khiển SoC của hãng Espressif, tích hợp Wi-Fi 2,4 GHz và Bluetooth, phù hợp với các ứng dụng IoT. ESP32 có hai lõi xử lý và hỗ trợ FreeRTOS, cho phép phân chia và thực hiện nhiều tác vụ đồng thời.

Thông số kỹ thuật chính:

- Vi xử lý: Xtensa LX6 32-bit, 2 lõi.
- Xung nhịp: Tối đa 240 MHz.
- Bộ nhớ SRAM: 520 KB.

- Kết nối không dây: Wi-Fi 802.11 b/g/n (2,4 GHz), Bluetooth.
- Điện áp hoạt động: 3,3 V.
- Hỗ trợ ngoại vi: ADC, DAC, PWM, UART, SPI, I2C.
- Hệ điều hành: FreeRTOS.

Trong bài Lab, ESP32 đảm nhiệm việc đọc cảm biến, xử lý dữ liệu, truyền thông MQTT và điều khiển máy bơm, van xả. Các chân GPIO sử dụng gồm:

- GPIO5, GPIO18: Kết nối cảm biến HC-SR04.
- GPIO4: Kết nối cảm biến DS18B20.
- GPIO26: Điều khiển relay máy bơm.
- GPIO27: Điều khiển relay van xả.

Nhờ khả năng xử lý hai lõi, ESP32 cho phép phân chia các Task FreeRTOS, giúp hệ thống giám sát và điều khiển hoạt động hiệu quả.

2.2. Cảm biến siêu âm HC-SR04

HC-SR04 là cảm biến siêu âm dùng để đo khoảng cách mà không cần tiếp xúc trực tiếp với vật thể. Trong đề tài, cảm biến được sử dụng để đo khoảng cách từ vị trí lắp đặt đến mặt nước, từ đó xác định mực nước trong bể nuôi thủy, hải sản. Cảm biến kết nối với ESP32 thông qua hai chân tín hiệu TRIG và ECHO.

Thông số của cảm biến:

Điện áp hoạt động: 5V.

Phạm vi đo danh định: 2–400 cm.

Độ chính xác điển hình: khoảng ±3 mm trong điều kiện phù hợp.

Giao tiếp: tín hiệu xung TRIG và ECHO.

Nguyên lý hoạt động:

ESP32 gửi tín hiệu kích hoạt đến chân TRIG, HC-SR04 phát sóng siêu âm hướng xuống mặt nước. Khi sóng phản xạ trở lại, cảm biến xuất tín hiệu tại chân ECHO để ESP32 xác định thời gian truyền sóng và tính khoảng cách đến mặt nước theo công thức:

$$d = \frac{t \times 0{,}0343}{2}$$

Trong đó:d là khoảng cách từ cảm biến đến mặt nước (cm), tlà thời gian sóng truyền đi và phản xạ trở lại (µs).

Dựa vào khoảng cách đo được và chiều cao bể, ESP32 tính mực nước hiện tại theo công thức:

$$\boxed{h = H - d}$$

Trong đó:h là chiều cao mực nước (cm), H là chiều cao bể (cm).

2.3. Cảm biến nhiệt độ DS18B20

DS18B20 là cảm biến nhiệt độ kỹ thuật số có khả năng đo nhiệt độ trong phạm vi rộng và truyền dữ liệu trực tiếp đến vi điều khiển thông qua giao thức 1-Wire. Trong đề tài, cảm biến được sử dụng để đo nhiệt độ nước trong bể nuôi thủy, hải sản, giúp người vận hành theo dõi thêm một thông số môi trường bên cạnh mực nước.

Thông số của cảm biến:

Điện áp hoạt động: khoảng 3,0–5,5V.

Phạm vi đo: từ -55°C đến +125°C.

Độ chính xác điển hình: ±0,5°C trong khoảng -10°C đến +85°C.

Độ phân giải: cấu hình được từ 9 đến 12 bit.

Giao tiếp: 1-Wire, sử dụng chân dữ liệu DQ.

Điện trở kéo lên: 4,7 kΩ.

Nguyên lý hoạt động:

DS18B20 sử dụng phần tử cảm biến nhiệt độ bên trong để xác định nhiệt độ môi trường tiếp xúc. Giá trị đo được chuyển đổi thành dữ liệu số ngay bên trong cảm biến. ESP32

giao tiếp với DS18B20 qua chân GPIO4 theo giao thức 1-Wire để yêu cầu đo và đọc dữ liệu nhiệt độ.

Điện trở 4,7 kΩ được mắc giữa chân DQ và nguồn 3,3V nhằm duy trì mức logic cao khi đường truyền không hoạt động. Sau khi nhận dữ liệu, ESP32 xử lý và hiển thị nhiệt độ để phục vụ việc giám sát điều kiện môi trường trong bể nuôi.

2.4. Cơ cấu chấp hành: máy bơm và van thoát

Hệ thống sử dụng hai cơ cấu chấp hành để điều chỉnh mực nước trong bể:
Máy bơm nước vào

- Chức năng: Bơm nước bổ sung vào bể khi mực nước thấp.
- Trong mô phỏng: Sử dụng module relay kết nối GPIO26 để mô phỏng trạng thái bật/tắt máy bơm.
- Thực tế: Sử dụng máy bơm DC hoặc AC, điều khiển thông qua relay hoặc MOSFET.

Van thoát nước

- Chức năng: Xả bớt nước khi mực nước vượt ngưỡng cho phép.

- Trong mô phỏng: Sử dụng module relay kết nối GPIO27 để mô phỏng trạng thái đóng/mở van.
- Thực tế: Sử dụng van điện từ 12V/24V, điều khiển thông qua relay hoặc MOSFET.

Nguyên lý điều khiển:
ESP32 xử lý dữ liệu mực nước từ cảm biến HC-SR04, sau đó xuất tín hiệu GPIO để điều khiển relay. Khi relay được kích hoạt, máy bơm hoặc van thoát hoạt động; khi relay ngắt, thiết bị dừng hoạt động.

III. HỆ ĐIỀU HÀNH THỜI GIAN THỰC RTOS VÀ FREERTOS

3.1. Khái niệm RTOS

RTOS (Real-Time Operating System) là hệ điều hành thời gian thực, được thiết kế để xử lý các tác vụ trong khoảng thời gian xác định. Đặc điểm quan trọng của RTOS là đảm bảo các tác vụ hoàn thành đúng thời hạn.
RTOS được chia thành hai loại:

- *Hard Real-Time:* Tác vụ bắt buộc phải hoàn thành đúng thời hạn, nếu không có thể gây hậu quả nghiêm trọng.
- *Soft Real-Time:* Việc chậm trễ có thể chấp nhận được nhưng làm giảm chất lượng hoạt động của hệ thống.

Hệ thống giám sát bể nuôi trong bài Lab thuộc nhóm Soft Real-Time.
Ưu điểm của RTOS

- Đa nhiệm: Cho phép nhiều Task hoạt động đồng thời.
- Đáp ứng nhanh: Task có mức ưu tiên cao được xử lý trước.
- Dễ quản lý: Chia chương trình thành các Task độc lập theo từng chức năng.
- Dễ mở rộng: Có thể bổ sung Task mới mà không phải thay đổi toàn bộ chương trình.
- Trao đổi dữ liệu: Hỗ trợ Queue, Semaphore, Mutex để đồng bộ giữa các Task.
- Tận dụng CPU: Khi một Task chờ dữ liệu, CPU có thể thực hiện Task khác.

So với lập trình vòng lặp thông thường (Super-loop), RTOS giúp phân chia công việc rõ ràng hơn, đặc biệt phù hợp với hệ thống vừa đọc cảm biến, vừa truyền dữ liệu và điều khiển thiết bị.

3.2. Giới thiệu FreeRTOS

FreeRTOS là hệ điều hành thời gian thực mã nguồn mở, được thiết kế dành cho các hệ thống nhúng và vi điều khiển có tài nguyên hạn chế.
FreeRTOS được tích hợp sẵn trên ESP32, cho phép lập trình viên tạo và quản lý nhiều Task để thực hiện các chức năng khác nhau.
Các thành phần chính gồm:

- Task: Đơn vị thực hiện một chức năng cụ thể.
- Scheduler: Bộ lập lịch, phân chia thời gian xử lý cho các Task.
- Queue: Hàng đợi dùng để trao đổi dữ liệu giữa các Task.
- Semaphore và Mutex: Cơ chế đồng bộ và bảo vệ tài nguyên dùng chung.
- Software Timer: Bộ định thời thực hiện các chức năng theo chu kỳ.

Ưu điểm của FreeRTOS

- Miễn phí, mã nguồn mở và có thể sử dụng trong các dự án thương mại.
- Nhỏ gọn, tiêu thụ ít tài nguyên.
- Hỗ trợ nhiều dòng vi điều khiển và kiến trúc xử lý.
- Dễ dàng tạo, quản lý và phân chia các Task.
- Cung cấp Queue, Semaphore, Mutex phục vụ trao đổi và đồng bộ dữ liệu.
- Được tích hợp sẵn trong ESP32, thuận tiện cho việc phát triển ứng dụng IoT.

Nhược điểm của FreeRTOS

- Cần thiết kế mức ưu tiên và dung lượng Stack phù hợp cho từng Task.
- Tiêu tốn thêm RAM để lưu trữ thông tin và Stack của các Task.
- Việc gỡ lỗi phức tạp hơn lập trình tuần tự.
- Có thể xảy ra xung đột dữ liệu, deadlock hoặc tràn Stack nếu lập trình không đúng cách.
- Không cung cấp sẵn đầy đủ hệ thống tệp và các dịch vụ như những hệ điều hành lớn.

3.3 FreeRTOS trên ESP32

ESP32 sử dụng kiến trúc hai lõi xử lý, hỗ trợ FreeRTOS để thực hiện nhiều tác vụ đồng thời. Một số đặc điểm quan trọng:

- xTaskCreatePinnedToCore(): Tạo Task và chỉ định lõi xử lý.
- vTaskDelay(): Tạm dừng Task trong một khoảng thời gian.
- xQueueCreate(): Khởi tạo hàng đợi trao đổi dữ liệu.
- xQueueSend(): Gửi dữ liệu vào Queue.
- xQueueReceive(): Nhận dữ liệu từ Queue.
- xQueueOverwrite(): Ghi đè dữ liệu trong Queue có độ dài bằng 1.
- Mức ưu tiên càng lớn thì Task càng được ưu tiên xử lý cao hơn.

Phân chia Task trong bài Lab:

- Core 0: Chạy Task truyền dữ liệu MQTT, đảm nhiệm kết nối Wi-Fi, gửi dữ liệu lên Web và nhận lệnh điều khiển.
- Core 1: Chạy Task đọc cảm biến và Task điều khiển, đảm nhiệm thu thập thông số môi trường và xử lý hoạt động của máy bơm, van xả.

Việc phân chia các Task trên hai lõi giúp tách biệt quá trình đọc cảm biến, điều khiển thiết bị và truyền thông mạng, góp phần nâng cao tính ổn định của hệ thống.

IV. CƠ CHẾ GIAO TIẾP GIỮA CÁC TASK

4.1. Nhu cầu giao tiếp giữa các Task

Khi chia chương trình thành nhiều Task, các Task không thể hoạt động hoàn toàn tách rời: Task đọc cảm biến phải chuyển kết quả đo cho Task truyền thông và Task điều khiển; Task truyền thông phải chuyển lệnh nhận từ Web cho Task điều khiển. Nếu dùng biến toàn cục để trao đổi, hai Task có thể cùng đọc/ghi một biến tại cùng thời điểm (tranh chấp dữ liệu –

race condition) hoặc Task nhận không biết khi nào dữ liệu mới sẵn sàng, buộc phải liên tục kiểm tra (polling) gây lãng phí CPU.

FreeRTOS cung cấp các cơ chế giao tiếp và đồng bộ được kernel quản lý, trong đó hai cơ chế cơ bản và quan trọng nhất là Hàng đợi (Queue) và Cờ hiệu (Semaphore).

4.2. Hàng đợi (Queue)

4.2.1. Khái niệm và nguyên lý hoạt động

Queue là vùng đệm theo nguyên tắc FIFO (vào trước – ra trước) dùng để truyền dữ liệu giữa các Task hoặc giữa ngắt và Task. Khi tạo Queue cần khai báo số phần tử tối đa và kích thước mỗi phần tử. Điểm quan trọng là Queue sao chép giá trị (copy by value) của phần tử vào vùng đệm của kernel: Task gửi có thể dùng lại hoặc xóa biến gốc ngay sau khi gửi mà không ảnh hưởng đến dữ liệu trong hàng đợi.

Task gửi và Task nhận có thể được cấu hình chờ (block) có thời gian giới hạn khi hàng đợi đầy hoặc rỗng. Task bị chặn không tiêu tốn CPU và sẽ tự động được đánh thức khi có dữ liệu hoặc có chỗ trống. Do đó, Queue vừa trao đổi dữ liệu vừa đồng bộ thời điểm hoạt động giữa các Task.

Các hàm thao tác chính với Queue

Hàm	**Chức năng**	**Ghi chú**
xQueueCreate(n, size)	Tạo hàng đợi	Trả về QueueHandle_t (NULL nếu thiếu bộ nhớ)
xQueueSend()/xQueueSendToBack()	Gửi phần tử vào cuối hàng đợi	Timeout = 0: không chờ, trả về lỗi nếu đầy
xQueueReceive()	Nhận và lấy phần tử ra khỏi hàng đợi	Block tối đa theo timeout
xQueuePeek()	Đọc phần tử đầu mà không xóa	
xQueueOverwrite()	Ghi đè phần tử trong hàng đợi dài 1	Luôn thành công; giữ giá trị mới nhất

Hàm	Chức năng	Ghi chú
xQueueSendFromISR()/xQueueReceiveFromISR()	Phiên bản dùng trong hàm ngắt	Không được chặn trong ISR

4.2.2. Ưu điểm

- An toàn luồng (thread-safe): kernel bảo vệ các thao tác gửi/nhận, không cần tự khóa.
- Truyền dữ liệu có cấu trúc: phần tử có thể là số, struct gồm nhiều trường (như SensorData, ControlCommand trong bài Lab).
- Đồng bộ ngầm: Task nhận chờ ở trạng thái Blocked, không tốn CPU cho đến khi có dữ liệu.
- Tách rời bên gửi và bên nhận: hai Task chạy với tốc độ khác nhau vẫn phối hợp được nhờ vùng đệm.
- Giữ thứ tự dữ liệu theo FIFO; có thể đặt độ dài hàng đợi để chống mất mẫu khi bên nhận chậm tạm thời.

4.2.3. Ứng dụng

- Chuyển dữ liệu cảm biến từ Task đo sang Task xử lý/truyền thông (như dataQueue).
- Gửi lệnh điều khiển từ Task giao tiếp (MQTT, UART, Bluetooth) sang Task điều khiển (như commandQueue).
- Hàng đợi sự kiện từ hàm ngắt (nút nhấn, encoder) sang Task xử lý.
- Hàng đợi chứa bản tin ghi log để một Task riêng ghi ra Serial hoặc thẻ nhớ.
- Hộp thư (mailbox) lưu giá trị mới nhất bằng hàng đợi dài 1 và xQueueOverwrite (như dataAlert, statusQueue).

4.3. Cờ hiệu (Semaphore) và Mutex

4.3.1. Khái niệm và các loại

Semaphore là cơ chế đồng bộ dùng để báo hiệu một sự kiện đã xảy ra hoặc để kiểm soát quyền truy cập tài nguyên dùng chung. Khác với Queue, Semaphore không mang theo dữ liệu mà chỉ mang giá trị đếm. Hai thao tác cơ bản là Give (nhả/tăng giá trị) và Take (lấy/giảm giá trị; nếu giá trị bằng 0 thì Task chờ).

Bảng 4.2. Các loại Semaphore trong FreeRTOS

Loại	Hàm tạo	Đặc điểm	Dùng khi
Binary Semaphore	xSemaphoreCreateBinary()	Giá trị 0 hoặc 1	Báo hiệu sự kiện từ ngắt sang Task, đồng bộ hai Task
Counting Semaphore	xSemaphoreCreateCounting(max, init)	Giá trị từ 0 đến max	Đếm số sự kiện, quản lý nhóm tài nguyên (ví dụ N bộ đệm)
Mutex	xSemaphoreCreateMutex()	Có chủ sở hữu, có kế thừa ưu tiên (priority inheritance)	Bảo vệ tài nguyên dùng chung: Serial, I2C, biến chung
Recursive Mutex	xSemaphoreCreateRecursiveMutex()	Cùng một Task có thể Take nhiều lần	Hàm lồng nhau cùng khóa một tài nguyên

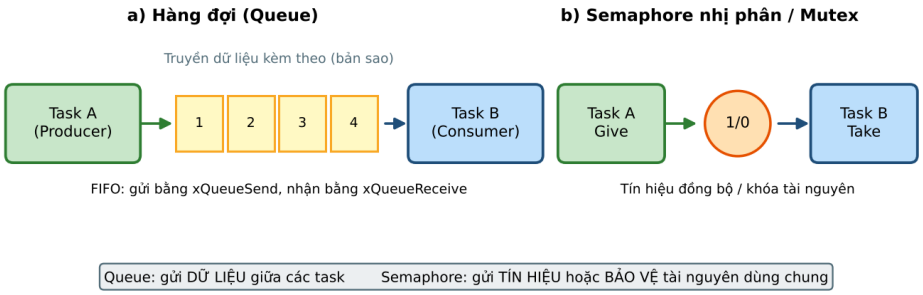
Mutex là loại Semaphore đặc biệt dành cho loại trừ tương hỗ (mutual exclusion). Task cần dùng tài nguyên phải Take mutex, dùng xong phải Give; trong thời gian đó Task khác muốn dùng sẽ phải chờ. Cơ chế kế thừa ưu tiên giúp giảm hiện tượng đảo ngược ưu tiên (priority inversion): Task ưu tiên thấp đang giữ mutex được tạm nâng lên mức ưu tiên của Task cao đang chờ, nhờ vậy không bị Task trung bình chen ngang.

4.3.2. Ưu điểm

- Báo hiệu sự kiện nhanh, chi phí thấp, đặc biệt hiệu quả để "đánh thức" Task từ hàm ngắt (xSemaphoreGiveFromISR).
- Bảo vệ vùng găng (critical section), tránh hai Task cùng truy cập tài nguyên không chia sẻ được (cổng Serial, bus I2C/SPI, biến trạng thái chung).
- Mutex hỗ trợ kế thừa ưu tiên, hạn chế đảo ngược ưu tiên.
- Counting Semaphore cho phép quản lý số lượng tài nguyên hữu hạn hoặc đếm sự kiện đến liên tiếp.

4.3.3. Ứng dụng

- Ngắt nút nhấn/cảm biến Give một binary semaphore để Task xử lý Take và thực hiện công việc.
- Dùng mutex bảo vệ cổng Serial khi nhiều Task cùng in thông tin để tránh chuỗi bị trộn lẫn.
- Dùng mutex bảo vệ các biến toàn cục chia sẻ giữa nhiều Task (ví dụ autoMode, pumpOn, valveOpen).
- Dùng counting semaphore giới hạn số Task truy cập đồng thời vào một tài nguyên (ví dụ nhóm kết nối).



Nguyên lý Queue và Semaphore/Mutex

4.4. Lựa chọn cơ chế giao tiếp trong bài Lab

Bài Lab sử dụng **Queue** làm cơ chế giao tiếp chính giữa các Task. Queue cho phép truyền dữ liệu có cấu trúc, đảm bảo các Task trao đổi thông tin mà không cần truy cập trực tiếp vào dữ liệu của nhau.
Hệ thống sử dụng 4 hàng đợi với chức năng cụ thể như sau:

Tên Queue	Hướng truyền dữ liệu	Kiểu dữ liệu	Số phần tử	Chức năng
dataQueue	TaskReadData → TaskTransmitData	SensorData	5	Truyền dữ liệu cảm biến phục vụ MQTT
dataAlert	TaskReadData → TaskAlert	float	1	Cập nhật mực nước mới nhất
commandQueue	TaskTransmitData → TaskAlert	ControlCommand	5	Truyền lệnh điều khiển từ Web
statusQueue	TaskAlert → TaskTransmitData	ActuatorStatus	1	Cập nhật trạng thái máy bơm và van xả

Nguyên lý hoạt động:

- dataQueue: Lưu tối đa 5 mẫu dữ liệu cảm biến, giúp tránh mất dữ liệu khi Task truyền thông chưa xử lý kịp.
- dataAlert: Chỉ lưu giá trị mực nước mới nhất, sử dụng xQueueOverwrite() để cập nhật dữ liệu.
- commandQueue: Tiếp nhận lệnh điều khiển từ Web thông qua MQTT và chuyển đến TaskAlert xử lý.
- statusQueue: Lưu trạng thái mới nhất của máy bơm và van xả để TaskTransmitData gửi lên Web.

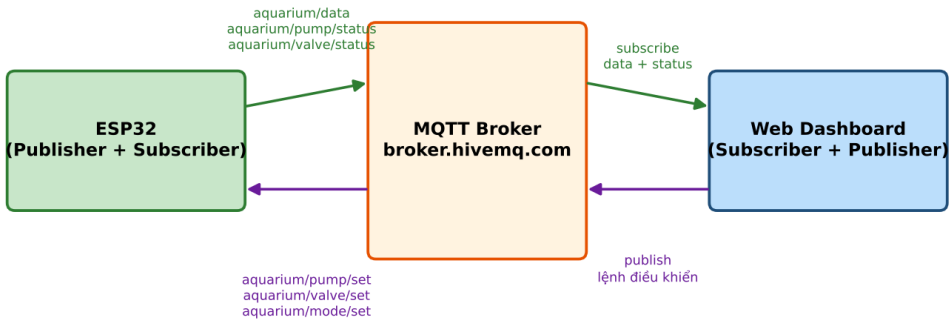
V. GIAO THỨC MQTT VÀ MQTT BROKER

5.1. Giới thiệu MQTT

MQTT (Message Queuing Telemetry Transport) là giao thức truyền thông điệp nhẹ, hoạt động trên TCP/IP theo mô hình Publish/Subscribe, do IBM đề xuất và hiện là chuẩn OASIS/ISO. MQTT được thiết kế cho thiết bị tài nguyên thấp, băng thông hẹp và đường truyền không ổn định nên được dùng rộng rãi trong IoT.

Các thành phần chính của MQTT:

- Publisher: thiết bị hoặc ứng dụng gửi (publish) thông điệp lên một topic.
- Subscriber: thiết bị hoặc ứng dụng đăng ký (subscribe) topic để nhận thông điệp.
- Broker: máy chủ trung gian nhận thông điệp từ publisher và chuyển tiếp đến các subscriber đã đăng ký topic tương ứng.
- Topic: chuỗi phân cấp bằng dấu "/" dùng để định tuyến, ví dụ aquarium/pump/set.



Mô hình Publish/Subscribe của hệ thống giám sát bể nuôi

Một số đặc tính quan trọng của MQTT:

- QoS (chất lượng dịch vụ): QoS 0 (tối đa một lần), QoS 1 (ít nhất một lần), QoS 2 (đúng một lần). Hệ thống dùng QoS 0 mặc định của thư viện PubSubClient vì dữ liệu cảm biến được gửi lặp lại liên tục.
- Retained message: Broker giữ lại bản tin cuối cùng của một topic để gửi ngay cho subscriber mới. Hai topic trạng thái bơm/van được gửi với cờ retained nên trang Web mở sau vẫn biết trạng thái hiện tại.
- Keep-alive: khoảng thời gian client báo còn sống với Broker.

Ưu điểm và ứng dụng của MQTT

- Nhẹ: tiêu đề cố định chỉ 2 byte, tiết kiệm băng thông và năng lượng.
- Mô hình Publish/Subscribe tách rời bên gửi và bên nhận, dễ mở rộng thêm thiết bị hoặc giao diện.
- Hỗ trợ nhiều mức QoS, retained message, Last Will để xử lý mất kết nối.
- Chạy ổn định trên mạng chập chờn; hỗ trợ bảo mật TLS và xác thực tài khoản.
- Có thư viện cho hầu hết nền tảng (Arduino PubSubClient, MQTT.js, Python Paho) và hỗ trợ WebSocket để dùng trực tiếp trên trình duyệt.

MQTT được ứng dụng trong nông nghiệp thông minh, nuôi trồng thủy sản, nhà thông minh, công nghiệp (IIoT), giao thông – logistics và y tế từ xa.

5.2 Các MQTT Broker thông dụng

Bảng 5.1. So sánh một số MQTT Broker thông dụng

Broker	Loại	Đặc điểm	Phù hợp
Eclipse Mosquitto	Mã nguồn mở, tự cài	Rất nhẹ, cài trên Raspberry Pi/PC	Mạng nội bộ, dự án nhỏ
EMQX	Mã nguồn mở / Cloud / Public	Hiệu năng cao, có Dashboard, broker công cộng broker.emqx.io	Học tập, hệ thống lớn
HiveMQ	Thương mại / Cloud / Public	Có broker công cộng broker.hivemq.com, bản Cloud miễn phí giới hạn	Doanh nghiệp, thử nghiệm
RabbitMQ (plugin MQTT)	Mã nguồn mở	Message broker đa giao thức	Hệ thống đã dùng RabbitMQ
AWS IoT Core	Dịch vụ đám mây	Tích hợp hệ sinh thái AWS, bảo mật cao, tính phí theo lưu lượng	Sản phẩm thương mại quy mô lớn

5.3 Lựa chọn EMQX Public Broker

Trong đề tài, EMQX Public Broker tại địa chỉ broker.emqx.io được lựa chọn làm máy chủ trung gian để truyền dữ liệu giữa ESP32 và giao diện giám sát. Việc sử dụng Broker công khai giúp đơn giản hóa quá trình cấu hình, không cần tự xây dựng máy chủ và thuận tiện cho việc mô phỏng, kiểm thử hệ thống.

ESP32 kết nối với Broker thông qua Wi-Fi và gửi dữ liệu cảm biến lên các Topic MQTT đã thiết lập. Giao diện giám sát đăng ký các Topic tương ứng để nhận thông tin về khoảng cách đến mặt nước, mực nước và nhiệt độ trong bể. Nếu triển khai điều khiển từ xa, các lệnh điều khiển máy bơm và van xả có thể được truyền qua những Topic riêng để ESP32 tiếp nhận và xử lý.

Ưu điểm:

Không cần cài đặt và vận hành máy chủ MQTT riêng.

Thuận tiện cho việc mô phỏng và thử nghiệm trên ESP32.

Hỗ trợ trao đổi dữ liệu giữa thiết bị và giao diện giám sát.

Có thể mở rộng để phục vụ chức năng điều khiển từ xa.

Nhược điểm:

EMQX Public Broker là máy chủ công khai, do đó các Topic dễ đoán có thể bị những người dùng khác truy cập hoặc gửi bản tin gây ảnh hưởng đến quá trình thử nghiệm. Ngoài ra, không nên mặc định rằng dịch vụ công khai bảo đảm tính sẵn sàng hoặc mức độ bảo mật cần thiết cho vận hành thực tế. Vì vậy, khi triển khai hệ thống thực tế cần sử dụng xác thực, phân quyền truy cập và kết nối mã hóa phù hợp.

VI.CÁC BƯỚC THỰC HIỆN BÀI LAB

6.1 Sơ đồ mạch trong Wokwi

Sơ đồ đấu nối

-Kết nối cảm biến HC-SR04 với ESP32

Chân HC-SR04	Chân ESP32	Chức năng
VCC	VIN (5V)	Cấp nguồn cho cảm biến
TRIG	GPIO 5	Phát tín hiệu kích hoạt đo
ECHO	GPIO 18	Nhận tín hiệu phản hồi siêu âm
GND	GND	Nối đất chung

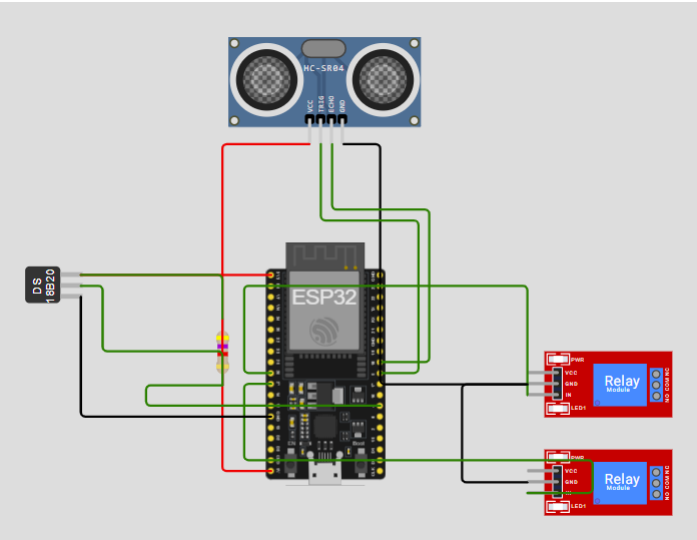
-Kết nối cảm biến DS18B20 với ESP32

Chân DS18B20	Chân ESP32	Chức năng
VCC	3V3	Cấp nguồn cho cảm biến
DQ	GPIO 4	Truyền dữ liệu nhiệt độ
GND	GND	Nối đất chung
Điện trở 4,7 kΩ	Giữa DQ và 3V3	Điện trở kéo lên cho đường dữ liệu

-Kết nối module relay với ESP32

Thiết bị	Chân module relay	Chân ESP32	Chức năng
Relay máy bơm	IN	GPIO 26	Điều khiển bật/tắt máy bơm
Relay máy bơm	VCC	VIN (5V)	Cấp nguồn cho module relay
Relay máy bơm	GND	GND	Nối đất chung
Relay van xả	IN	GPIO 27	Điều khiển bật/tắt van xả
Relay van xả	VCC	VIN (5V)	Cấp nguồn cho module relay
Relay van xả	GND	GND	Nối đất chung

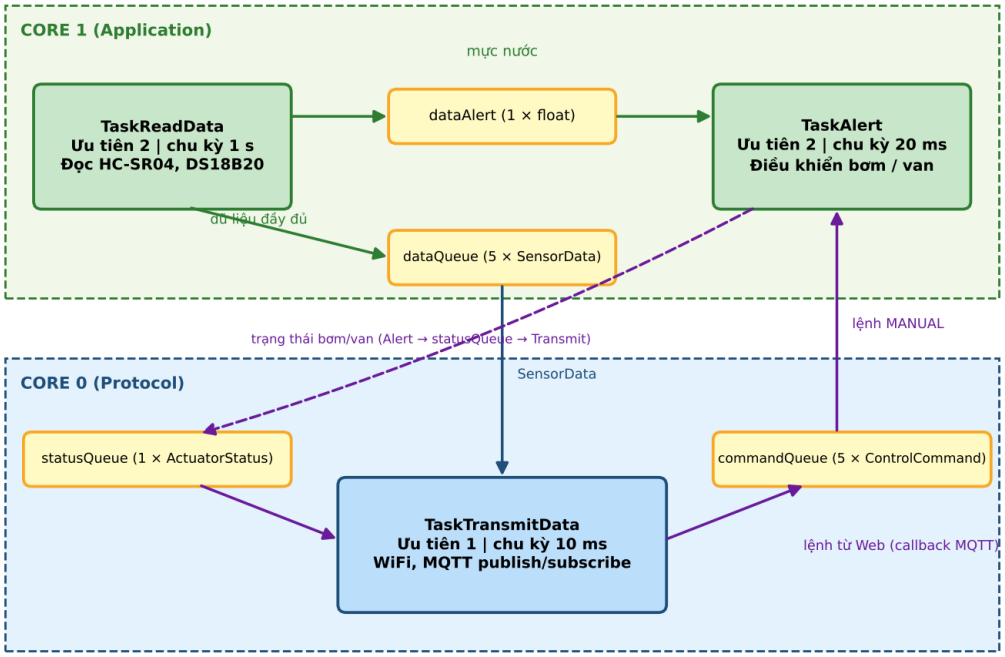
Mô phỏng trên Wokwi



6.2 Triển khai FreeRTOS trên ESP32

6.2.1. Phân chia Task và các hàng đợi

Chương trình được chia thành ba Task theo đúng yêu cầu của đề bài. Sơ đồ dưới đây thể hiện vị trí của các Task trên hai lõi và hướng dữ liệu đi qua các hàng đợi.



Sơ đồ Task, hàng đợi và dòng dữ liệu của hệ thống

Bảng dữ liệu cấu hình các Task

Thuộc tính	TaskReadData	TaskTransmitData	TaskAlert
Chức năng	Đọc dữ liệu cảm biến	Truyền thông MQTT	Giám sát và điều khiển
Lõi chạy (Core)	1	0	1
Đầu vào	HC-SR04, DS18B20	dataQueue, MQTT	dataAlertQueue, cmdQueue
Đầu ra	Serial, dataQueue, dataAlertQueue	MQTT, cmdQueue	Relay bơm, relay van
Nhiệm vụ	Đọc khoảng cách, nhiệt độ và tính mực nước	Kết nối Wi-Fi, gửi dữ liệu lên Broker, nhận lệnh từ Web	Xử lý dữ liệu, điều khiển bơm và van theo chế độ Auto/Manual

Lý do phân chia như trên:

TaskReadData và TaskAlert chạy trên Core 1, đảm nhiệm việc đọc cảm biến và xử lý điều khiển.

TaskTransmitData chạy trên Core 0, đảm nhiệm kết nối Wi-Fi và truyền nhận dữ liệu MQTT.

Các Task trao đổi dữ liệu thông qua Queue, hạn chế việc truy cập trực tiếp vào dữ liệu của nhau.

6.2.2. Bảng dữ liệu của các Task và hàng đợi

Cấu trúc dữ liệu truyền giữa các Task

Kiểu dữ liệu	Trường	Kiểu	Ý nghĩa
SensorData	distance	float	Khoảng cách từ cảm biến đến mặt nước (cm)
	waterLevel	float	Mực nước trong bể (cm)
	temperature	float	Nhiệt độ bể (°C)
ControlCommand	device	char[]	Thiết bị nhận lệnh: pump, valve hoặc mode
	value	char[]	Giá trị lệnh: ON, OFF, AUTO hoặc MANUAL

Bảng các hàng đợi (Queue)

Tên Queue	Hướng truyền dữ liệu	Kiểu dữ liệu	Chức năng
dataQueue	TaskReadData → TaskTransmitData	SensorData	Truyền dữ liệu cảm biến
dataAlertQueue	TaskReadData → TaskAlert	float	Cập nhật mực nước
cmdQueue	TaskTransmitData → TaskAlert	ControlCommand	Truyền lệnh điều khiển

6.3Bảng topic và cổng kết nối

Danh sách topic MQTT

Topic	Hướng truyền	Kiểu dữ liệu	Nội dung
aquatank/demo_83921/data	ESP32 → Web	JSON	Dữ liệu cảm biến
aquatank/demo_83921/cmd/mode	Web → ESP32	Chuỗi	Chọn chế độ AUTO/MANUAL
aquatank/demo_83921/cmd/pump	Web → ESP32	Chuỗi	Điều khiển máy bơm
aquatank/demo_83921/cmd/valve	Web → ESP32	Chuỗi	Điều khiển van xả
aquatank/demo_83921/status/pump	ESP32 → Web	Chuỗi	Trạng thái máy bơm
aquatank/demo_83921/status/valve	ESP32 → Web	Chuỗi	Trạng thái van xả
aquatank/demo_83921/status/mode	ESP32 → Web	Chuỗi	Trạng thái chế độ

Thông số kết nối của từng thành phần

Thành phần	Giao thức	Thông số kết nối
ESP32 → Wi-Fi	Wi-Fi 2,4 GHz	Wokwi-GUEST
ESP32 → Broker	MQTT/TCP	broker.emqx.io:1883
Web → Broker	MQTT over WebSocket	broker.emqx.io:8084
ESP32 → Serial	UART	115200 baud

6.4. Thuật toán điều khiển mực nước

Hệ thống có hai chế độ hoạt động là Auto và Manual.

Chế độ Auto

Ở chế độ Auto, TaskAlert căn cứ vào dữ liệu mực nước nhận từ dataAlertQueue để điều khiển máy bơm và van xả.

Logic điều khiển theo mực nước

Điều kiện mực nước	Hành động	Thời gian
Mực nước dưới 50 cm	Mở van xả	10 giây
Mực nước từ 100 đến dưới 150 cm	Bật máy bơm theo mức 50% mô phỏng	10 giây
Mực nước từ 150 đến 200 cm	Bật máy bơm theo mức 100% mô phỏng	20 giây
Các vùng còn lại	Không tác động	—

Chế độ Manual

Ở chế độ Manual, người dùng điều khiển trực tiếp máy bơm và van xả thông qua các nút trên giao diện Web. Lệnh được truyền qua MQTT đến ESP32, sau đó TaskAlert tiếp nhận và thực hiện điều khiển.

Nguyên tắc hoạt động

- TaskAlert liên tục kiểm tra dữ liệu mực nước để xác định trạng thái điều khiển.
- Khi mực nước thay đổi sang vùng cần xử lý, hệ thống thực hiện hành động tương ứng.
- Máy bơm và van xả được điều khiển độc lập theo lệnh của chương trình.
- Khi chuyển sang chế độ Manual, việc điều khiển tự động được vô hiệu hóa.
- Trạng thái thiết bị được cập nhật và gửi lên Web thông qua MQTT.

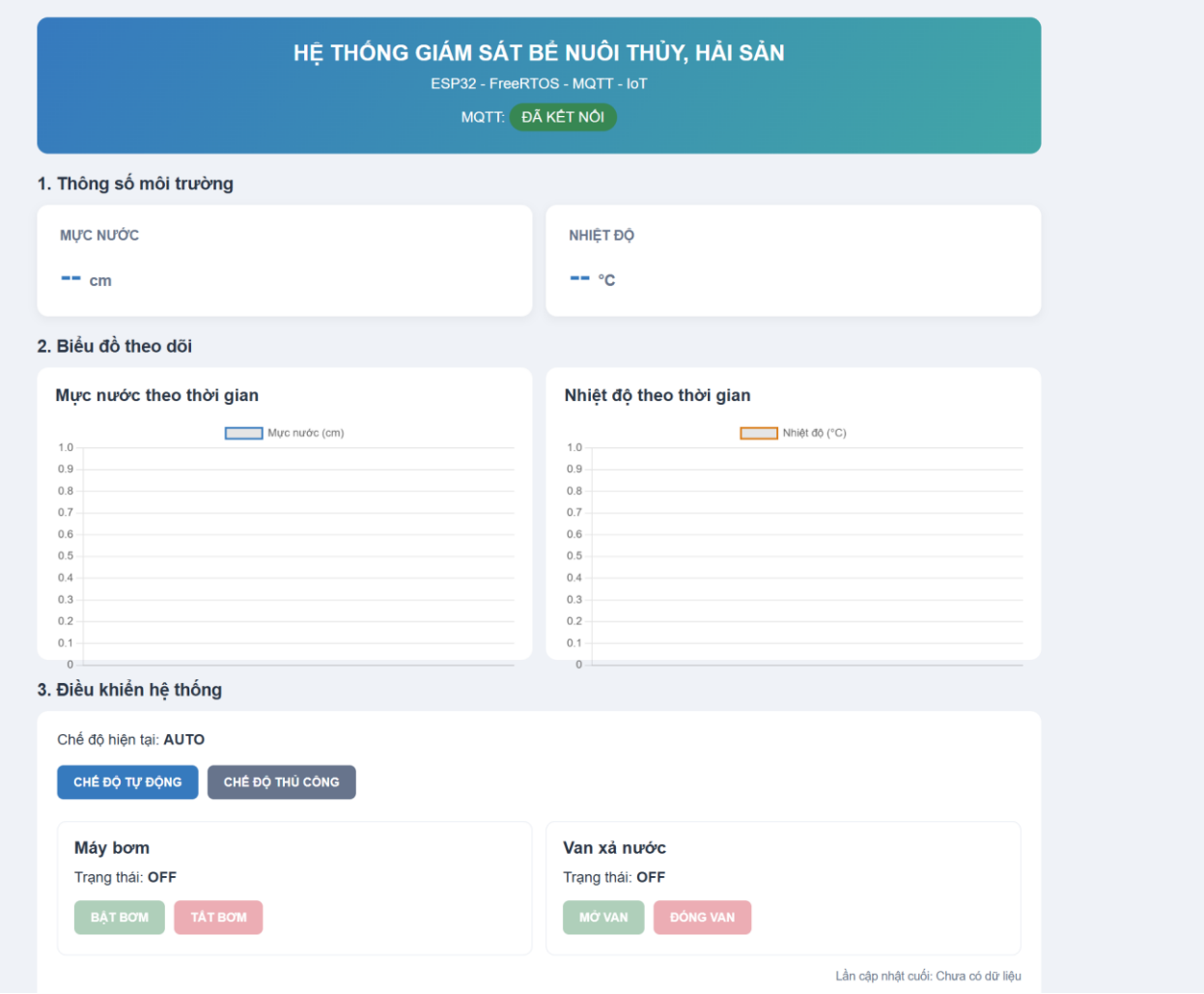
6.5. Giao diện Web

Giao diện Web được xây dựng bằng HTML, sử dụng thư viện MQTT.js để kết nối với MQTT Broker EMQX thông qua WebSocket và Chart.js để hiển thị biểu đồ thời gian

thực.Giao diện được thiết kế trong file html, cho phép người dùng theo dõi thông số môi trường và điều khiển thiết bị từ xa.

Các thành phần giao diện

Thành phần	Thẻ HTML / id	Chức năng
Hiển thị thông số	`level`, `temperature`	Hiển thị mực nước và nhiệt độ bể.
Biểu đồ mực nước	`levelChart`	Theo dõi mực nước theo thời gian.
Biểu đồ nhiệt độ	`temperatureChart`	Theo dõi nhiệt độ theo thời gian.
Chọn chế độ	`changeMode()`	Chuyển đổi AUTO/MANUAL.
Điều khiển máy bơm	`pumpOn`, `pumpOff`	Bật/tắt máy bơm ở chế độ MANUAL.
Điều khiển van xả	`valveOn`, `valveOff`	Mở/đóng van xả ở chế độ MANUAL.
Hiển thị trạng thái	`pumpStatus`, `valveStatus`, `mode`, `mqttStatus`, `lastUpdate`	Cập nhật trạng thái thiết bị, chế độ, kết nối và thời gian nhận dữ liệu.



Bố cục giao diện Web giám sát và điều khiển

Cơ chế điều khiển máy bơm và van xả: Các nút điều khiển chỉ được kích hoạt khi Web kết nối thành công với MQTT Broker và hệ thống đang ở chế độ MANUAL. Khi chuyển sang AUTO hoặc mất kết nối Broker, các nút sẽ bị vô hiệu hóa. Trạng thái máy bơm và van xả chỉ được cập nhật khi nhận phản hồi từ ESP32 qua các topic MQTT trạng thái, giúp giao diện hiển thị đúng trạng thái thực tế của thiết bị.

6.6 Mã nguồn và giải thích

6.6.1. Chương trình ESP32

a) Khai báo thư viện, cấu hình Wi-Fi, MQTT và chân GPIO

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <OneWire.h>
#include <DallasTemperature.h>
```

```
// Wi-Fi
const char* WIFI_SSID = "Wokwi-GUEST";
const char* WIFI_PASS = "";

// MQTT
const char* MQTT_HOST = "broker.emqx.io";
const int MQTT_PORT = 1883;
const char* ROOT_TOPIC = "aquatank/demo_83921";

String TOPIC_DATA = String(ROOT_TOPIC) + "/data";
String TOPIC_MODE = String(ROOT_TOPIC) + "/cmd/mode";
String TOPIC_PUMP = String(ROOT_TOPIC) + "/cmd/pump";
String TOPIC_VALVE = String(ROOT_TOPIC) + "/cmd/valve";

String TOPIC_PUMP_STATE = String(ROOT_TOPIC) + "/status/pump";
String TOPIC_VALVE_STATE = String(ROOT_TOPIC) + "/status/valve";
String TOPIC_MODE_STATE = String(ROOT_TOPIC) + "/status/mode";

// GPIO
#define TRIG_PIN 5
#define ECHO_PIN 18
#define TEMP_PIN 4
#define PUMP_RELAY 26
#define VALVE_RELAY 27

#define RELAY_ON HIGH
#define RELAY_OFF LOW

#define TANK_HEIGHT_CM 200.0

// MQTT OBJECT
WiFiClient espClient;
PubSubClient mqtt(espClient);
```

```
// DS18B20
OneWire oneWire(TEMP_PIN);
DallasTemperature sensors(&oneWire);
```

Chương trình sử dụng bốn thư viện chính: WiFi.h để kết nối Wi-Fi, PubSubClient.h để giao tiếp MQTT, OneWire.h và DallasTemperature.h để đọc dữ liệu từ cảm biến DS18B20.

Hệ thống kết nối Broker EMQX thông qua địa chỉ broker.emqx.io, cổng 1883. Các topic được chia thành ba nhóm: gửi dữ liệu cảm biến, nhận lệnh điều khiển và gửi trạng thái thiết bị.

Các chân GPIO được khai báo tương ứng với cảm biến và relay. Chiều cao bể được thiết lập là 200 cm, relay kích mức HIGH để bật và LOW để tắt.

b) Cấu trúc dữ liệu, hàng đợi và biến trạng thái

```
struct SensorData {
  float distance;
  float level;
  float temperature;
};

enum CommandType {
  CMD_MODE,
  CMD_PUMP,
  CMD_VALVE
};

struct WebCommand {
  CommandType type;
  bool value;
};

// QUEUE
QueueHandle_t dataQueue;
QueueHandle_t dataAlertQueue;
QueueHandle_t cmdQueue;

// DEVICE STATE
volatile bool pumpState = false;
volatile bool valveState = false;
volatile bool autoMode = true;
```

Chương trình sử dụng cấu trúc SensorData để lưu ba thông số gồm khoảng cách, mực nước và nhiệt độ. Cấu trúc WebCommand lưu loại lệnh và giá trị điều khiển nhận từ Web.c) Các hàm điều khiển thiết bị và cập nhật trạng thái

c) Các hàm đọc cảm biến và điều khiển thiết bị

```
float readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(3);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration =
    pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0)
    return -1;

  return duration * 0.0343 / 2;
}

float calculateLevel(float distance) {
  if (distance < 0)
    return -1;

  float level = TANK_HEIGHT_CM - distance;

  if (level < 0)
    level = 0;

  if (level > TANK_HEIGHT_CM)
```

```
        level = TANK_HEIGHT_CM;

    return level;
}

// RELAY CONTROL
void controlPump(bool state) {
    pumpState = state;
    digitalWrite(
        PUMP_RELAY,
        state ? RELAY_ON : RELAY_OFF
    );
}

void controlValve(bool state) {
    valveState = state;
    digitalWrite(
        VALVE_RELAY,
        state ? RELAY_ON : RELAY_OFF
    );
}
```

Hàm readDistance() tạo xung siêu âm tại chân TRIG và đo thời gian phản hồi tại chân ECHO.

Hàm calculateLevel() tính mực nước bằng cách lấy chiều cao bể trừ đi khoảng cách từ cảm biến đến mặt nước. Giá trị được giới hạn trong khoảng 0–200 cm.

Hai hàm controlPump() và controlValve() thực hiện bật/tắt máy bơm và van xả thông qua relay, đồng thời cập nhật trạng thái thiết bị.

d) Hàm cập nhật trạng thái MQTT

```
void publishStatus() {
    if (!mqtt.connected())
        return;
```

```
    mqtt.publish(
        TOPIC_PUMP_STATE.c_str(),
        pumpState ? "ON" : "OFF",
        true
    );

    mqtt.publish(
        TOPIC_VALVE_STATE.c_str(),
        valveState ? "ON" : "OFF",
        true
    );

    mqtt.publish(
        TOPIC_MODE_STATE.c_str(),
        autoMode ? "AUTO" : "MANUAL",
        true
    );
}
```

Hàm publishStatus() gửi trạng thái máy bơm, van xả và chế độ hoạt động lên MQTT Broker. Nếu MQTT chưa kết nối, hàm sẽ không thực hiện gửi dữ liệu.

Các bản tin sử dụng cờ retained (true) để Broker lưu lại trạng thái mới nhất. Nhờ đó, giao diện Web có thể nhận trạng thái hiện tại khi đăng ký topic.

e) Callback MQTT và kết nối Broker

```
void mqttCallback(
    char* topic,
    byte* payload,
    unsigned int length
) {
    String message = "";

    for (unsigned int i = 0; i < length; i++) {
        message += (char)payload[i];
```

```
}

message.trim();
message.toUpperCase();

WebCommand cmd;
bool valid = true;

String receivedTopic = String(topic);

if (receivedTopic == TOPIC_MODE) {
   cmd.type = CMD_MODE;

   if (message == "AUTO")
      cmd.value = true;
   else if (message == "MANUAL")
      cmd.value = false;
   else
      valid = false;
}
else if (receivedTopic == TOPIC_PUMP) {
   cmd.type = CMD_PUMP;

   if (message == "ON")
      cmd.value = true;
   else if (message == "OFF")
      cmd.value = false;
   else
      valid = false;
}
else if (receivedTopic == TOPIC_VALVE) {
```

```
        cmd.type = CMD_VALVE;

        if (message == "ON")
            cmd.value = true;
        else if (message == "OFF")
            cmd.value = false;
        else
            valid = false;
    }
    else {
        valid = false;
    }

    if (valid && cmdQueue != NULL) {
        xQueueSend(cmdQueue, &cmd, 0);
    }
}
bool connectMQTT() {
    if (mqtt.connected())
        return true;

    String clientID =
        "ESP32-AquaTank-" +
        String((uint32_t)ESP.getEfuseMac(), HEX);

    if (mqtt.connect(clientID.c_str())) {
        mqtt.subscribe(TOPIC_MODE.c_str());
        mqtt.subscribe(TOPIC_PUMP.c_str());
        mqtt.subscribe(TOPIC_VALVE.c_str());

        publishStatus();
```

```
      Serial.println("[MQTT] Connected!");
      return true;
    }

    return false;
}
```

Hàm mqttCallback() được gọi khi ESP32 nhận được bản tin từ MQTT Broker. Chương trình chuyển nội dung bản tin thành chữ in hoa, kiểm tra topic và xác định loại lệnh tương ứng.

Sau khi kiểm tra hợp lệ, lệnh được đưa vào cmdQueue để TaskAlert xử lý. Callback không trực tiếp điều khiển relay, giúp tách biệt quá trình nhận lệnh và điều khiển thiết bị.

Hàm connectMQTT() thực hiện kết nối Broker, đăng ký ba topic nhận lệnh và gửi trạng thái hiện tại của hệ thống khi kết nối thành công.

f) Task 1: TaskReadData – đọc dữ liệu cảm biến (Core 1)

```
void TaskReadData(void *parameter) {
    SensorData data;

    for (;;) {
      // Đọc HC-SR04
      data.distance = readDistance();

      // Đọc DS18B20
      sensors.requestTemperatures();

      data.temperature =
        sensors.getTempCByIndex(0);

      if (data.temperature == DEVICE_DISCONNECTED_C) {
        data.temperature = -127;
      }
```

```
// Tính mực nước
data.level = calculateLevel(data.distance);

// Hiển thị Serial
Serial.println("\n===== DU LIEU BE NUOI =====");

Serial.printf(
   "Khoang cach: %.1f cm\n",
   data.distance
);

Serial.printf(
   "Muc nuoc: %.1f cm\n",
   data.level
);

Serial.printf(
   "Nhiet do: %.1f C\n",
   data.temperature
);

Serial.println("==========================");

// Gửi dữ liệu vào data Queue
if (xQueueSend(dataQueue, &data, 0) != pdPASS) {
   SensorData oldData;

   xQueueReceive(dataQueue, &oldData, 0);
   xQueueSend(dataQueue, &data, 0);
}
```

```
      // Gửi mực nước vào Alert Queue
      if (data.level >= 0) {
        xQueueOverwrite(
          dataAlertQueue,
          &data.level
        );
      }

      vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

TaskReadData chạy trên Core 1 với chu kỳ 1 giây. Nhiệm vụ chính là đọc dữ liệu từ hai cảm biến HC-SR04 và DS18B20, sau đó tính toán mực nước và hiển thị các thông số trên Serial Monitor.

Dữ liệu cảm biến được gửi vào dataQueue để TaskTransmitData truyền lên MQTT. Nếu Queue đầy, chương trình loại bỏ dữ liệu cũ nhất để ưu tiên dữ liệu mới.

Đồng thời, mực nước được gửi vào dataAlertQueue bằng xQueueOverwrite() để TaskAlert luôn nhận được giá trị cập nhật gần nhất.

g) Task 2: TaskTransmitData – truyền dữ liệu MQTT (Core 0)

```
void TaskTransmitData(void *parameter) {
  SensorData data;
  char json[256];

  unsigned long lastStatus = 0;

  for (;;) {
    // Kết nối WiFi
    if (WiFi.status() != WL_CONNECTED) {
      WiFi.begin(WIFI_SSID, WIFI_PASS);

      while (WiFi.status() != WL_CONNECTED) {
        vTaskDelay(pdMS_TO_TICKS(500));
```

```
    }

    Serial.println("[WiFi] Connected!");
    Serial.println(WiFi.localIP());
}

// Kết nối MQTT
if (!mqtt.connected()) {
    connectMQTT();
}

if (mqtt.connected()) {
    mqtt.loop();

    // Đọc dữ liệu từ data Queue
    if (xQueueReceive(
        dataQueue,
        &data,
        pdMS_TO_TICKS(100)
    ) == pdPASS) {

        snprintf(
            json,
            sizeof(json),
            "{\"distance_cm\":%.1f,"
            "\"level_cm\":%.1f,"
            "\"temperature_c\":%.1f,"
            "\"pump\":\"%s\","
            "\"valve\":\"%s\","
            "\"mode\":\"%s\"}",
```

```
                data.distance,
                data.level,
                data.temperature,
                pumpState ? "ON" : "OFF",
                valveState ? "ON" : "OFF",
                autoMode ? "AUTO" : "MANUAL"
            );

            mqtt.publish(
                TOPIC_DATA.c_str(),
                json
            );

            Serial.print("[MQTT] ");
            Serial.println(json);
        }

        // Cập nhật trạng thái định kỳ
        if (millis() - lastStatus >= 2000) {
            publishStatus();
            lastStatus = millis();
        }
    }

    vTaskDelay(pdMS_TO_TICKS(20));
  }
}
```

TaskTransmitData chạy trên Core 0, đảm nhiệm việc kết nối Wi-Fi, MQTT và truyền dữ liệu cảm biến lên Broker.

Khi có dữ liệu trong dataQueue, chương trình sử dụng snprintf() để đóng gói các thông số thành chuỗi JSON, sau đó gửi lên topic /data.

Ngoài ra, Task sử dụng mqtt.loop() để duy trì kết nối và xử lý bản tin MQTT. Trạng thái máy bơm, van xả và chế độ hoạt động được cập nhật định kỳ 2 giây thông qua hàm publishStatus()

h) Task 3: TaskAlert – điều khiển tự động và thủ công (Core 1)

Phần 1: Nhận và xử lý lệnh từ Web.

```
void TaskAlert(void *parameter) {
  float latestLevel = -1;
  float receivedLevel;

  WebCommand cmd;

  enum Action {
    IDLE,
    PUMP_50,
    PUMP_100,
    VALVE_OPEN
  };

  Action action = IDLE;

  unsigned long actionStart = 0;
  unsigned long lastToggle = 0;

  bool pumpHalfCycle = false;
  int previousZone = 0;

  for (;;) {
    // Nhận lệnh điều khiển từ Web
    while (xQueueReceive(cmdQueue, &cmd, 0) == pdPASS) {

      if (cmd.type == CMD_MODE) {
        autoMode = cmd.value;
```

```
    action = IDLE;

    controlPump(false);
    controlValve(false);

    previousZone = 0;

    Serial.println(
      autoMode ? "MODE AUTO" : "MODE MANUAL"
    );
}

else if (cmd.type == CMD_PUMP && !autoMode) {
    action = IDLE;

    if (cmd.value) {
      controlValve(false);
      controlPump(true);
    }
    else {
      controlPump(false);
    }
}

else if (cmd.type == CMD_VALVE && !autoMode) {
    action = IDLE;

    if (cmd.value) {
      controlPump(false);
      controlValve(true);
```

```
            }
            else {
                controlValve(false);
            }
        }
    }
```

Phần 2: Điều khiển tự động theo mực nước.

```
    // Đọc mực nước từ Alert Queue
    if (xQueueReceive(
        dataAlertQueue,
        &receivedLevel,
        0
    ) == pdPASS) {
        latestLevel = receivedLevel;
    }

    unsigned long now = millis();

    // AUTO MODE
    if (autoMode && latestLevel >= 0) {
        int currentZone = 0;

        if (latestLevel >= 100 && latestLevel < 150) {
            currentZone = 1;
        }
        else if (latestLevel >= 150 && latestLevel <= 200) {
            currentZone = 2;
        }
        else if (latestLevel < 50) {
            currentZone = 3;
        }
```

```
// Chỉ kích hoạt khi đi vào vùng mới
if (currentZone != previousZone &&
    currentZone != 0 &&
    action == IDLE) {

    actionStart = now;
    lastToggle = now;

    controlPump(false);
    controlValve(false);

    if (currentZone == 1) {
        action = PUMP_50;
        pumpHalfCycle = true;
        controlPump(true);

        Serial.println(
            "AUTO: BOM 50% - 10 GIAY"
        );
    }
    else if (currentZone == 2) {
        action = PUMP_100;
        controlPump(true);

        Serial.println(
            "AUTO: BOM 100% - 20 GIAY"
        );
    }
    else if (currentZone == 3) {
        action = VALVE_OPEN;
```

```
            controlValve(true);

            Serial.println(
               "AUTO: MO VAN XA - 10 GIAY"
            );
        }
    }

    previousZone = currentZone;
}

// TIMED CONTROL
if (action == PUMP_50) {
    if (now - actionStart >= 10000) {
        controlPump(false);
        action = IDLE;

        Serial.println("KET THUC BOM 50%");
    }
    else if (now - lastToggle >= 1000) {
        lastToggle = now;
        pumpHalfCycle = !pumpHalfCycle;
        controlPump(pumpHalfCycle);
    }
}

else if (action == PUMP_100) {
    if (now - actionStart >= 20000) {
        controlPump(false);
        action = IDLE;
```

```
            Serial.println("KET THUC BOM 100%");
        }
    }

    else if (action == VALVE_OPEN) {
        if (now - actionStart >= 10000) {
            controlValve(false);
            action = IDLE;

            Serial.println("DONG VAN XA");
        }
    }

    vTaskDelay(pdMS_TO_TICKS(50));
  }
}
```

i) Hàm setup() và loop(): Khởi tạo hệ thống và tạo Task

```
void setup() {
  Serial.begin(115200);

  // GPIO
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(PUMP_RELAY, OUTPUT);
  pinMode(VALVE_RELAY, OUTPUT);

  controlPump(false);
  controlValve(false);

  // Khởi tạo cảm biến
  sensors.begin();
```

```
// Tạo Queue
dataQueue = xQueueCreate(
    10,
    sizeof(SensorData)
);

dataAlertQueue = xQueueCreate(
    1,
    sizeof(float)
);

cmdQueue = xQueueCreate(
    10,
    sizeof(WebCommand)
);

if (dataQueue == NULL ||
    dataAlertQueue == NULL ||
    cmdQueue == NULL) {

    Serial.println("ERROR: CREATE QUEUE FAILED!");

    while (true) {
        delay(1000);
    }
}

// Cấu hình MQTT
mqtt.setServer(MQTT_HOST, MQTT_PORT);
mqtt.setCallback(mqttCallback);
```

```
mqtt.setBufferSize(512);

// Task MQTT - Core 0
xTaskCreatePinnedToCore(
    TaskTransmitData,
    "TaskTransmitData",
    8192,
    NULL,
    2,
    NULL,
    0
);

// Task đọc cảm biến - Core 1
xTaskCreatePinnedToCore(
    TaskReadData,
    "TaskReadData",
    4096,
    NULL,
    2,
    NULL,
    1
);

// Task điều khiển - Core 1
xTaskCreatePinnedToCore(
    TaskAlert,
    "TaskAlert",
    4096,
    NULL,
    2,
```

```
        NULL,
        1
    );

    Serial.println("HE THONG DA KHOI DONG!");
}

void loop() {
    vTaskDelay(pdMS_TO_TICKS(1000));
}
```

Hàm setup() thực hiện khởi tạo Serial, cấu hình các chân GPIO, cảm biến DS18B20 và MQTT. Sau đó, chương trình tạo ba Queue với kích thước phù hợp với từng loại dữ liệu.

Tiếp theo, hàm xTaskCreatePinnedToCore() tạo ba Task và phân bổ lên hai lõi ESP32:

- Core 0: TaskTransmitData.
- Core 1: TaskReadData và TaskAlert.

Các Task có mức ưu tiên bằng 2. Stack được cấp phát lần lượt 8192 byte và 4096 byte theo cấu hình chương trình.

Hàm loop() không thực hiện xử lý chính mà sử dụng vTaskDelay() để nhường CPU cho các Task FreeRTOS.

6.6.2. Chương trình Web

a) Nạp thư viện và khai báo topic

```
<script src="https://unpkg.com/mqtt/dist/mqtt.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

const ROOT_TOPIC = "aquatank/demo_83921";

const TOPIC_DATA = ROOT_TOPIC + "/data";
const TOPIC_MODE = ROOT_TOPIC + "/cmd/mode";
const TOPIC_PUMP = ROOT_TOPIC + "/cmd/pump";
const TOPIC_VALVE = ROOT_TOPIC + "/cmd/valve";
```

```
const TOPIC_PUMP_STATE = ROOT_TOPIC + "/status/pump";
const TOPIC_VALVE_STATE = ROOT_TOPIC + "/status/valve";
const TOPIC_MODE_STATE = ROOT_TOPIC + "/status/mode";

let connected = false;
let mode = "AUTO";
let pumpOn = false;
let valveOn = false;

const MAX_POINTS = 30;
```

b) Tạo biểu đồ thời gian thực

```
function makeChart(id, label, color) {
   return new Chart(document.getElementById(id), {
      type: "line",
      data: {
         labels: [],
         datasets: [{
            label: label,
            data: [],
            borderColor: color,
            backgroundColor: color + "33",
            fill: true,
            tension: 0.3,
            pointRadius: 2
         }]
      },
      options: {
         animation: false,
         responsive: true,
         scales: {
            x: {
```

```
                ticks: { maxTicksLimit: 6 }
            }
        }
    }
  });
}

const levelChart = makeChart(
  "levelChart", "Mực nước (cm)", "#0277bd"
);

const temperatureChart = makeChart(
  "temperatureChart", "Nhiệt độ (°C)", "#c62828"
);

function addPoint(chart, value) {
  chart.data.labels.push(
    new Date().toLocaleTimeString("vi-VN")
  );

  chart.data.datasets[0].data.push(value);

  if (chart.data.labels.length > MAX_POINTS) {
    chart.data.labels.shift();
    chart.data.datasets[0].data.shift();
  }

  chart.update("none");
```

Hàm makeChart() khởi tạo biểu đồ đường để hiển thị sự thay đổi của mực nước và nhiệt độ theo thời gian.

Hàm addPoint() bổ sung giá trị mới cùng thời gian nhận dữ liệu vào biểu đồ. Hệ thống chỉ lưu tối đa 30 điểm gần nhất, tự động loại bỏ dữ liệu cũ khi vượt giới hạn. Lệnh chart.update("none") giúp cập nhật biểu đồ mà không sử dụng hiệu ứng chuyển động.

}c) Kết nối Broker qua WebSocket

```
const client = mqtt.connect(
  "wss://broker.emqx.io:8084/mqtt",
  {
    clientId: "aquatank_web_" +
      Math.random().toString(16).slice(2, 10),

    clean: true,
    connectTimeout: 10000,
    reconnectPeriod: 3000
  }
);

function setConnection(text, cls) {
  const el = document.getElementById("mqttStatus");
  el.textContent = text;
  el.className = "badge " + cls;
}

client.on("connect", function () {
  connected = true;

  setConnection("ĐÃ KẾT NỐI", "ok");

  client.subscribe([
    TOPIC_DATA,
    TOPIC_PUMP_STATE,
    TOPIC_VALVE_STATE,
```

```
        TOPIC_MODE_STATE
    ]);

    updateUI();
});

client.on("reconnect", function () {
    connected = false;

    setConnection("ĐANG KẾT NỐI LẠI", "warn");

    updateUI();
});

client.on("close", function () {
    connected = false;

    setConnection("MẤT KẾT NỐI", "bad");

    updateUI();
});

client.on("error", function () {
    setConnection("LỖI KẾT NỐI", "bad");
});
```

Web kết nối đến MQTT Broker EMQX thông qua giao thức MQTT over WebSocket Secure (WSS), sử dụng cổng 8084.

clientId được tạo ngẫu nhiên nhằm hạn chế trùng lặp giữa các phiên kết nối. Tham số reconnectPeriod = 3000 cho phép trình duyệt tự động thử kết nối lại sau mỗi 3 giây khi mất kết nối.

Khi kết nối thành công, Web đăng ký bốn topic để nhận dữ liệu cảm biến và trạng thái thiết bị. Các sự kiện connect, reconnect, close và error được sử dụng để cập nhật trạng thái kết nối trên giao diện.

d) Nhận dữ liệu và cập nhật thẻ số, biểu đồ, trạng thái

```
client.on("message", function (topic, payload) {
  const text = payload.toString().trim();

  if (topic === TOPIC_DATA) {
    let d;

    try {
      d = JSON.parse(text);
    } catch (e) {
      return;
    }

    if (!Number.isFinite(d.level_cm) ||
      !Number.isFinite(d.temperature_c)) {
      return;
    }

    document.getElementById("level").textContent =
      d.level_cm.toFixed(1);

    document.getElementById("temperature").textContent =
      d.temperature_c.toFixed(1);

    addPoint(levelChart, d.level_cm);
    addPoint(temperatureChart, d.temperature_c);

    document.getElementById("lastUpdate").textContent =
      new Date().toLocaleTimeString("vi-VN");
  }
```

```
  else if (topic === TOPIC_PUMP_STATE) {
    pumpOn = (text === "ON");
    updateUI();
  }

  else if (topic === TOPIC_VALVE_STATE) {
    valveOn = (text === "ON");
    updateUI();
  }

  else if (topic === TOPIC_MODE_STATE) {
    mode = text;
    updateUI();
  }
});
```

Khi nhận được bản tin MQTT, chương trình kiểm tra topic để xác định loại dữ liệu cần xử lý.

Đối với topic /data, dữ liệu JSON được phân tích bằng JSON.parse(). Hai thông số level_cm và temperature_c được cập nhật lên giao diện, đồng thời bổ sung vào biểu đồ tương ứng.

Đối với các topic trạng thái, Web cập nhật biến pumpOn, valveOn và mode theo phản hồi thực tế từ ESP32. Nhờ đó, giao diện luôn đồng bộ với trạng thái hệ thống.

e) Gửi lệnh điều khiển bằng nút bấm

```
function send(topic, message) {
  if (!connected) return;

  client.publish(topic, message, { qos: 0 });
}
```

```
function updateUI() {
  document.getElementById("mode").textContent = mode;

  document.getElementById("pumpStatus").textContent =
    pumpOn ? "ĐANG BẬT" : "ĐANG TẮT";

  document.getElementById("valveStatus").textContent =
    valveOn ? "ĐANG MỞ" : "ĐANG ĐÓNG";

  const canControl = connected && mode === "MANUAL";

  const pumpBtn = document.getElementById("pumpOn");
  const pumpOffBtn = document.getElementById("pumpOff");

  const valveBtn = document.getElementById("valveOn");
  const valveOffBtn = document.getElementById("valveOff");

  pumpBtn.disabled = !canControl;
  pumpOffBtn.disabled = !canControl;

  valveBtn.disabled = !canControl;
  valveOffBtn.disabled = !canControl;
}

function changeMode(value) {
  send(TOPIC_MODE, value);
}

function controlPump(value) {
  if (!connected || mode !== "MANUAL") return;
```

```
  send(TOPIC_PUMP, value);
}

function controlValve(value) {
  if (!connected || mode !== "MANUAL") return;

  send(TOPIC_VALVE, value);
}
```

Hàm send() kiểm tra kết nối MQTT trước khi gửi lệnh đến ESP32.

Các hàm changeMode(), controlPump() và controlValve() lần lượt gửi lệnh chuyển chế độ, bật/tắt máy bơm và mở/đóng van xả thông qua các topic tương ứng.

Các nút điều khiển máy bơm và van xả chỉ được kích hoạt khi Web đã kết nối Broker và hệ thống đang ở chế độ MANUAL. Trạng thái và nhãn thiết bị được cập nhật dựa trên phản hồi từ ESP32 qua các topic /status/, tránh việc giao diện tự hiển thị trạng thái chưa được xác nhận.

KẾT LUẬN

Bài Lab đã thiết kế thành công hệ thống tự động giám sát thông số môi trường cho bể nuôi thủy, hải sản dùng ESP32, cảm biến siêu âm HC-SR04 và cảm biến nhiệt độ DS18B20. Dữ liệu mực nước và nhiệt độ được hiển thị qua Serial, truyền qua MQTT Broker HiveMQ và hiển thị theo thời gian thực trên giao diện Web gồm hai thẻ số, hai biểu đồ và các nút điều khiển máy bơm, van thoát ở chế độ Manual.

Chương trình được xây dựng theo kiến trúc FreeRTOS với ba Task (đọc dữ liệu, truyền dữ liệu, cảnh báo) phân bổ lên hai lõi của ESP32 và giao tiếp với nhau qua bốn hàng đợi. Thuật toán điều khiển theo ba vùng mực nước (bơm 50% trong 10 s, bơm 100% trong 20 s, mở van 10 s) hoạt động theo sự kiện đổi vùng, có giới hạn thời gian và có chế độ Manual ưu tiên.

Qua bài Lab, em nắm được ưu nhược điểm của RTOS/FreeRTOS, cách dùng Queue và Semaphore để giao tiếp giữa các Task, cách kết nối thiết bị IoT với MQTT Broker và xây dựng giao diện Web điều khiển từ xa. Hệ thống có thể mở rộng thành giải pháp giám sát thực tế khi bổ sung mạch công suất, cảm biến chất lượng nước, broker bảo mật và lưu trữ dữ liệu lâu dài.

TÀI LIỆU THAM KHẢO

1. FreeRTOS – Real-time operating system for microcontrollers, tài liệu chính thức: https://www.freertos.org
2. Espressif Systems, ESP-IDF Programming Guide – FreeRTOS (IDF) và Arduino-ESP32 documentation: https://docs.espressif.com
3. Espressif Systems, ESP32 Series Datasheet.
4. Elecfreaks, HC-SR04 Ultrasonic Ranging Module – Datasheet.
5. Maxim Integrated (Analog Devices), DS18B20 Programmable Resolution 1-Wire Digital Thermometer – Datasheet.
6. OASIS, MQTT Version 3.1.1 / 5.0 Specification.
7. HiveMQ, Public MQTT Broker: https://www.hivemq.com/mqtt/public-mqtt-broker/
8. Nick O'Leary, PubSubClient – Arduino Client for MQTT: https://github.com/knolleary/pubsubclient

9. MQTT.js – MQTT client for Node.js and the browser: https://github.com/mqttjs/MQTT.js
10. Chart.js – Simple yet flexible JavaScript charting: https://www.chartjs.org
11. Wokwi – Online Arduino and ESP32 Simulator, tài liệu: https://docs.wokwi.com